

Programação Orientada a Objetos em Python

Curso completo — do zero à aplicação prática

Material de estudo — Edy / Cleitin

Agosto de 2026

Sumário

Como usar este curso	3
Capítulo 1 — Por que Orientação a Objetos	4
Conceito	4
Analogia	4
Os quatro pilares (mapa geral)	4
Erros comuns	5
Exercícios	5
Capítulo 2 — Classes e Objetos	6
Conceito	6
Atributos de instância vs atributos de classe	6
Erros comuns	7
Exercícios	7
Capítulo 3 — O construtor <code>__init__</code> e métodos de instância	8
Conceito	8
Analogia	8
<code>self</code> não é palavra reservada	8
Valores padrão e <code>*args/**kwargs</code> no construtor	9
Erros comuns	9
Exercícios	9
Capítulo 4 — Métodos de classe e métodos estáticos	11
Conceito	11
Analogia	11
Quando usar cada um	12
Erros comuns	12
Exercícios	12
Capítulo 5 — Encapsulamento (visão geral)	13
Conceito	13
Analogia	13
Exercícios (prévia — resolva os completos no documento de aprofundamento)	14
Capítulo 6 — Herança	15
Conceito	15
Analogia	15
<code>super()</code>	15
Herança múltipla e MRO	16
Erros comuns	16
Exercícios	16

Capítulo 7 — Polimorfismo (visão geral)	17
Conceito	17
Analogia	17
Exercícios (prévia)	18
Capítulo 8 — Abstração e classes abstratas (ABC)	19
Conceito	19
Analogia	19
Erros comuns	20
Exercícios	20
Capítulo 9 — Métodos mágicos (dunder methods)	21
Conceito	21
Tabela de referência rápida	21
Erros comuns	22
Exercícios	22
Capítulo 10 — Composição vs Herança	23
Conceito	23
Analogia	23
Quando usar cada um	23
Erros comuns	24
Exercícios	24
Capítulo 11 — Princípios SOLID (aplicados, sem enrolação)	25
Conceito	25
Por que isso importa aqui	25
Exercícios	25
Capítulo 12 — Projeto integrador	26
Especificação	26
Como validar seu próprio trabalho	26
Capítulo 13 — Trilha de estudos e próximos passos	27
Marco de progresso	27
Depois do aprofundamento em Encapsulamento e Polimorfismo, próximos marcos sugeridos	27
Reavaliação periódica	27
Apêndice — Gabarito comentado	28
Capítulo 2	28
Capítulo 3	28
Capítulo 4	29
Capítulo 6	29
Capítulo 7	30
Capítulo 9	30

Como usar este curso

Isso aqui não é um livro pra ler passivamente. É um treino. E treino que você só observa não gera hipertrofia nenhuma — o princípio é o mesmo pra código: **você fixa o que você pratica, não o que você lê.**

Estrutura de cada capítulo:

1. **Conceito** — a ideia central, direto ao ponto, sem enrolação.
2. **Analogia** — a maioria vai usar o domínio de treino/fitness, porque é onde sua intuição já é forte. Se você já entende progressão de carga, você já entende herança.
3. **Exemplo comentado** — código real, comentado no ponto que importa.
4. **Erros comuns** — o que a galera erra e por quê.
5. **Exercícios** — em ordem crescente de dificuldade. **Tente resolver antes de olhar qualquer solução.** As soluções ficam no apêndice no final do documento, separadas por capítulo — não do lado do exercício, de propósito. Se você for direto na resposta, você tá fazendo cardio de vista, não treino de força mental.

Método de estudo recomendado (baseado em evidência, não em achismo):

- **Active recall:** depois de ler o conceito, feche o material e tente explicar com suas palavras antes de ver o exemplo.
- **Prática espaçada:** não tenta consumir o curso inteiro num dia. Um capítulo, prática, descanso (tipo treino — hipertrofia acontece no descanso, aprendizado consolida no sono).
- **Progressive overload:** os exercícios de cada capítulo aumentam em dificuldade. Não pula pro difícil sem passar pelo fácil — você vai construir bug em cima de bug.
- **Falha controlada:** erro é dado, não é vergonha. Rode o código, quebre, leia o traceback até o fim antes de sair copiando stack overflow.

Este documento cobre a base completa de POO. Encapsulamento e Polimorfismo aparecem aqui em visão geral (capítulos 5 e 7) porque fazem parte do mapa completo — mas você pediu aprofundamento nesses dois pontos especificamente, então tem um segundo documento só pra isso, com mais profundidade, mais exercícios e um projeto prático integrador.

Capítulo 1 — Por que Orientação a Objetos

Conceito

Programação Orientada a Objetos (POO) é um paradigma que organiza o código em torno de **objetos**: entidades que combinam **dados** (atributos) e **comportamento** (métodos) em uma única unidade.

A alternativa mais comum antes disso — programação estruturada/procedural — separa dados e funções. Você tem uma `lista_de_atletas` e uma função `calcular_imc(atleta)` que opera nela. Funciona, mas cresce mal: conforme o sistema aumenta, fica difícil garantir que os dados estão sempre num estado válido, porque qualquer função em qualquer lugar do programa pode mexer neles.

POO resolve isso amarrando dado e comportamento: o objeto é responsável por si mesmo.

Analogia

Pensa num atleta com uma ficha de treino. Você não anda com uma planilha solta (dados) e uma pessoa aleatória decidindo o que fazer com ela (função solta). O atleta é a entidade: ele tem atributos (peso, altura, nível) e comportamentos (treinar, descansar, avaliar progresso). A ficha de treino não flutua sozinha manipulada por qualquer um — ela pertence ao atleta e só muda através de ações que fazem sentido pra ele.

Os quatro pilares (mapa geral)

Pilar	Resumo em uma frase
Encapsulamento	Esconder detalhes internos e expor só o que é necessário, protegendo o estado do objeto.
Herança	Reaproveitar e especializar comportamento de uma classe em outra.
Polimorfismo	Objetos diferentes respondem à mesma “mensagem” (método) de formas diferentes.
Abstração	Modelar só o que importa pro problema, ignorando detalhes irrelevantes.

Vamos construir cada um desses na ordem que faz sentido pedagogicamente — não necessariamente na ordem da tabela.

Erros comuns

- Achar que POO é “sobre classes”. Não é. É sobre **organizar responsabilidade**. Dá pra escrever POO ruim cheio de classes e dá pra escrever código procedural bem organizado. Classe é ferramenta, não objetivo.
- Criar uma classe pra tudo, inclusive coisas que são só uma função. Se não tem estado pra guardar, provavelmente não precisa ser classe.

Exercícios

1.1 — Sem escrever código ainda: liste 3 “objetos” do mundo real do seu domínio (pode ser fitness, pode ser o seu chatbot, pode ser o gerenciador de livros). Pra cada um, liste 2 atributos e 2 comportamentos.

1.2 — Pegue uma função procedural qualquer que você já escreveu (pode ser do Cleitinho_ChatBot) que recebe um dicionário e faz algo com ele. Escreva, em texto, como isso ficaria se fosse um objeto — sem código, só o raciocínio.

Capítulo 2 — Classes e Objetos

Conceito

Classe é o molde. **Objeto** (ou instância) é a coisa construída a partir do molde.

Isso é literalmente igual a “plano de treino padrão” (classe) vs “o treino específico do João hoje” (instância, com valores concretos: João pesa 82kg, treina hoje às 18h).

```
class Atleta:
    pass

# Instanciando – cada chamada cria um objeto novo e independente
joao = Atleta()
maria = Atleta()

print(joao is maria) # False – objetos diferentes, mesmo molde
```

Atributos de instância vs atributos de classe

```
class Atleta:
    esporte_padrao = "musculação" # atributo de CLASSE – compartilhado

    def __init__(self, nome, peso):
        self.nome = nome # atributo de INSTÂNCIA – próprio de cada objeto
        self.peso = peso

joao = Atleta("João", 82)
maria = Atleta("Maria", 65)

print(joao.esporte_padrao) # musculação
print(maria.esporte_padrao) # musculação – mesmo valor, é compartilhado

Atleta.esporte_padrao = "crossfit"
print(joao.esporte_padrao) # crossfit – mudou pra ambos, pois é da classe
```

Ponto de atenção crítico: se o atributo de classe for um objeto **mutável** (lista, dicionário), todas as instâncias compartilham a *mesma* referência — e isso é uma fonte clássica de bug.

```
class AtletaComBug:
    treinos = [] # PERIGO: lista compartilhada entre TODAS as instâncias

    def __init__(self, nome):
```

```

        self.nome = nome

a = AtletaComBug("João")
b = AtletaComBug("Maria")
a.treinos.append("agachamento")
print(b.treinos)  # ['agachamento'] – vazou pro Maria! Bug clássico.

```

A correção é sempre inicializar mutáveis dentro de `__init__`, como atributo de instância (capítulo 3 e 4 vão aprofundar isso).

Erros comuns

- Confundir `Atleta` (a classe) com `joao` (uma instância). `Atleta.peso` não existe até que algum objeto tenha esse atributo de instância.
- Usar atributo de classe mutável pensando que é “valor padrão por instância” — não é, é compartilhado.

Exercícios

2.1 — Crie uma classe `Exercicio` vazia (só `pass`). Instancie três exercícios diferentes (`supino`, `agachamento`, `remada`) e comprove com `is` que são objetos distintos.

2.2 — Reproduza o bug da lista compartilhada acima, mas com uma classe `Equipe` que tem `membros = []` como atributo de classe. Depois de reproduzir o bug, não conserte ainda — só descreva por escrito por que aconteceu.

2.3 (desafio) — Usando `Atleta.__dict__` e `joao.__dict__`, investigue a diferença entre o que fica guardado na classe e o que fica guardado na instância. O que você observa quando você faz `joao.peso = 90` depois de já ter `peso` definido no `__init__`?

Capítulo 3 — O construtor `__init__` e métodos de instância

Conceito

`__init__` é o método chamado automaticamente quando você instancia um objeto. Ele não “cria” o objeto (isso é trabalho do `__new__`, que você raramente vai mexer) — ele **inicializa** o objeto que já foi criado.

`self` é a referência ao próprio objeto sendo manipulado. Todo método de instância recebe `self` como primeiro parâmetro — o Python passa isso automaticamente quando você chama `objeto.metodo()`.

```
class Atleta:
    def __init__(self, nome, peso, altura):
        self.nome = nome
        self.peso = peso
        self.altura = altura
        self.historico_treinos = [] # mutável, inicializado por instância – certo

    def calcular_imc(self):
        return self.peso / (self.altura ** 2)

    def registrar_treino(self, treino):
        self.historico_treinos.append(treino)
        print(f"{self.nome} registrou: {treino}")

joao = Atleta("João", 82, 1.78)
print(round(joao.calcular_imc(), 1)) # 25.9
joao.registrar_treino("Peito e tríceps")
```

Analogia

`__init__` é a ficha de anamnese do dia 1 na academia: você não começa a treinar sem passar antes pela avaliação que registra peso, altura, objetivo. Sem isso, o personal (seus métodos) não tem dado nenhum pra trabalhar.

`self` não é palavra reservada

Por convenção (PEP 8) todo mundo usa `self`, mas o interpretador não liga pro nome — é só o primeiro parâmetro posicional do método, preenchido automaticamente pela sintaxe

objeto.metodo(args), que por baixo dos panos é Classe.metodo(objeto, args).

```
joao.calcular_imc()          # forma normal
Atleta.calcular_imc(joao)    # exatamente equivalente
```

Isso importa entender porque explica *por que* métodos de instância sempre recebem pelo menos um argumento.

Valores padrão e *args/**kwargs no construtor

```
class Atleta:
    def __init__(self, nome, peso, altura, nivel="iniciante"):
        self.nome = nome
        self.peso = peso
        self.altura = altura
        self.nivel = nivel

maria = Atleta("Maria", 65, 1.65)          # nivel = "iniciante" por padrão
carlos = Atleta("Carlos", 90, 1.80, "avançado")
```

Erros comuns

- Esquecer self na assinatura do método e tomar `TypeError: metodo() takes 0 positional arguments but 1 was given`.
- Usar valor mutável como **valor padrão** de parâmetro (`def __init__(self, treinos=[])`) — isso reproduz o mesmo bug do atributo de classe mutável, porque a lista padrão é criada **uma vez só**, na definição da função, e compartilhada entre chamadas que não passam o argumento.

```
# ERRADO
class AtletaBuggy:
    def __init__(self, nome, treinos=[]):
        self.nome = nome
        self.treinos = treinos

a = AtletaBuggy("João")
a.treinos.append("supino")
b = AtletaBuggy("Maria")
print(b.treinos)  # ['supino'] – vazou de novo!
```

```
# CERTO
class AtletaCorreto:
    def __init__(self, nome, treinos=None):
        self.nome = nome
        self.treinos = treinos if treinos is not None else []
```

Exercícios

3.1 — Crie uma classe `Exercicio` com `__init__(self, nome, grupo_muscular, series, repeticoes)`. Adicione um método `resumo(self)` que retorna uma string formatada tipo "Supino reto – Peito – 4x10".

3.2 — Crie uma classe PlanoDeTreino com um construtor que recebe nome_atleta e inicializa uma lista vazia exercicios. Adicione um método adicionar_exercicio(self, exercicio) que dá append na lista, e um método carga_total(self) que retorna a soma de series * repeticoes de todos os exercícios (use a classe Exercicio do exercício anterior).

3.3 (desafio — pegadinha do valor padrão mutável) — Escreva de propósito uma classe com o bug de lista mutável como valor padrão, comprove o bug com um teste (crie dois objetos e mostre que o estado vaza), depois corrija. Documente as duas versões lado a lado com um comentário explicando exatamente o motivo do bug.

Capítulo 4 — Métodos de classe e métodos estáticos

Conceito

Além de métodos de instância (o padrão, que recebem `self`), Python tem dois outros tipos:

- **@classmethod**: recebe a classe (`cls`) em vez da instância. Útil para *construtores alternativos* ou operações que dizem respeito à classe como um todo.
- **@staticmethod**: não recebe nem `self` nem `cls`. É basicamente uma função normal “morando dentro” da classe porque tematicamente pertence a ela.

```
from datetime import date

class Atleta:
    def __init__(self, nome, ano_nascimento):
        self.nome = nome
        self.ano_nascimento = ano_nascimento

    @classmethod
    def a_partir_de_idade(cls, nome, idade):
        """Construtor alternativo: cria um Atleta a partir da idade, não do ano."""
        ano_nascimento = date.today().year - idade
        return cls(nome, ano_nascimento) # cls(...) == Atleta(...)

    @staticmethod
    def imc_esta_saudavel(imc):
        """Não depende de nenhuma instância específica — é uma regra geral."""
        return 18.5 <= imc <= 24.9

joao = Atleta.a_partir_de_idade("João", 30)
print(joao.ano_nascimento) # 1996 (dependendo do ano atual)
print(Atleta.imc_esta_saudavel(23.5)) # True — chamado direto na classe
```

Analogia

`@classmethod` é tipo ter formas diferentes de “matricular” um atleta na academia: pelo RG, pelo CPF, por indicação — no final todas chegam no mesmo cadastro (`cls(...)`), só variam a porta de entrada. `@staticmethod` é a tabela de IMC saudável pendurada na parede: é uma regra da academia, não pertence a nenhum atleta específico, mas faz sentido ela morar ali porque é sobre o assunto “atleta/saúde”.

Quando usar cada um

Situação	Use
Precisa acessar/modificar <code>self.atributo</code> de uma instância específica	método de instância
Cria um objeto de um jeito alternativo, ou opera sobre a classe toda	<code>@classmethod</code>
Função utilitária relacionada ao tema da classe, mas sem depender de estado	<code>@staticmethod</code>

Erros comuns

- Usar `@staticmethod` quando na verdade precisava de `cls` ou `self` — sinal de que a função deveria ser método normal ou de classe.
- Esquecer que dentro de um `@classmethod`, `cls(...)` respeita herança (se uma subclasse chamar o `classmethod` herdado, `cls` é a subclasse, não a classe original) — isso é proposital e poderoso, mas surpreende quem não sabe.

Exercícios

4.1 — Na classe `Exercicio` do capítulo 3, adicione um `@classmethod` chamado `isometria(cls, nome, grupo_muscular, tempo_segundos)` que cria um exercício isométrico (prancha, por exemplo), representando `series=1` e `repeticoes` como o tempo em segundos.

4.2 — Adicione um `@staticmethod` em `Atleta` chamado `classificar_imc(imc)` que retorna uma string: "abaixo do peso", "peso normal", "sobrepeso" ou "obesidade", conforme as faixas padrão de IMC.

4.3 (desafio) — Crie uma classe `Equipe` com um `@classmethod` `a_partir_de_csv(cls, caminho_arquivo)` (pode simular com uma lista de strings em vez de ler arquivo de verdade) que faz parsing de linhas "nome,peso,altura" e retorna uma instância de `Equipe` já populada com Atletas. Isso é um padrão *muito* comum em código real.

Capítulo 5 — Encapsulamento (visão geral)

Este capítulo é o resumo. O aprofundamento completo — com @property, validação, name mangling e projeto prático — está no documento separado “Encapsulamento e Polimorfismo — Aprofundamento”, já que foi o ponto onde você parou.

Conceito

Encapsulamento é sobre **controlar o acesso ao estado interno de um objeto**, para garantir que ele nunca fique num estado inválido. Em vez de deixar qualquer código externo alterar `atleta.peso = -50` livremente, você expõe uma interface controlada.

Python não tem `private` de verdade como Java/C++. Usa convenção:

```
class Atleta:
    def __init__(self, nome, peso):
        self.nome = nome           # público – acesso livre
        self._peso = peso          # "protegido" – convenção, não bloqueio real
        self.__id_interno = 42    # "privado" – name mangling, dificulta acesso accidental

    @property
    def peso(self):
        return self._peso

    @peso.setter
    def peso(self, valor):
        if valor <= 0:
            raise ValueError("Peso precisa ser positivo")
        self._peso = valor
```

Isso garante que **não existe jeito de criar um Atleta com peso negativo** passando pelo caminho normal — o objeto se protege sozinho.

Analogia

É a diferença entre a régua da balança calibrada pela própria academia (só o técnico autorizado mexe, garantindo que a leitura é confiável) e qualquer aluno mexendo no calibre por conta própria. O `_peso` é a balança; `@property` é o técnico autorizado controlando o acesso.

Exercícios (prévia — resolva os completos no documento de aprofundamento)

5.1 — Sem usar `@property` ainda, crie uma classe `ContaTreino` com atributo `_saldo_creditos` e métodos `adicionar_creditos(self, qtd)` e `usar_credito(self)` que impedem o saldo de ficar negativo.

Capítulo 6 — Herança

Conceito

Herança permite criar uma classe nova (**subclasse**) a partir de uma existente (**superclasse**), reaproveitando atributos e métodos, e especializando ou sobrescrevendo o que for necessário.

```
class Atleta:
    def __init__(self, nome, peso):
        self.nome = nome
        self.peso = peso

    def apresentar(self):
        return f"{self.nome}, {self.peso}kg"

class Powerlifter(Atleta):
    def __init__(self, nome, peso, total_powerlifting):
        super().__init__(nome, peso) # reaproveita o __init__ da superclasse
        self.total_powerlifting = total_powerlifting

    def apresentar(self):
        base = super().apresentar() # reaproveita e estende
        return f"{base} - Total: {self.total_powerlifting}kg"

joao = Powerlifter("João", 90, 650)
print(joao.apresentar()) # João, 90kg - Total: 650kg
```

Analogia

Powerlifter **é um** Atleta — todo powerlifter tem nome e peso, mas tem também algo específico (total nas três provas). Herança modela relação “é um” (is-a). É diferente de “tem um” (has-a), que é composição (capítulo 10) — atleta *tem* uma ficha de treino, mas ficha de treino não é um atleta.

super()

`super()` te dá acesso aos métodos da superclasse sem precisar nomear a classe explicitamente — importante porque, em hierarquias com múltiplos níveis, isso evita que você tenha que reescrever `Atleta.__init__(self, ...)` manualmente (o que quebra fácil se a hierarquia mudar).

Herança múltipla e MRO

Python permite herdar de mais de uma classe. A ordem de resolução de método (**MRO** — **Method Resolution Order**) decide qual `__init__`/método é chamado quando há ambiguidade.

```
class Nadador:
    def treinar(self):
        return "nadando"

class Corredor:
    def treinar(self):
        return "correndo"

class Triatleta(Nadador, Corredor):
    pass

t = Triatleta()
print(t.treinar())          # "nadando" — Nadador vem primeiro na lista de herança
print(Triatleta.__mro__)    # mostra a ordem completa de resolução
```

Herança múltipla é poderosa mas fonte comum de confusão — em geral, prefira composição quando a relação não é claramente “é um”.

Erros comuns

- Esquecer `super().__init__(...)` na subclasse — os atributos da superclasse simplesmente não existem no objeto.
- Herança “porque sim”, sem relação real “é um”. Se você tá herdando só pra reaproveitar código sem que a relação semântica exista, o certo é composição.
- Sobrescrever um método e esquecer de manter o comportamento base quando ele ainda é necessário (esquecer o `super().metodo()`).

Exercícios

6.1 — Crie `Atleta` com nome, peso e método `treinar()` que retorna “treino genérico”. Crie `Nadador(Atleta)` e `Halterofilista(Atleta)`, cada um sobrescrevendo `treinar()` com uma mensagem específica.

6.2 — Adicione um atributo `nivel` em `Atleta` com valor padrão “iniciante”. Nas subclasses, use `super().__init__(...)` corretamente e adicione um atributo específico de cada modalidade (`estilo_preferido` para `Nadador`, `recorde_agachamento` para `Halterofilista`).

6.3 (desafio) — Construa a hierarquia `Triatleta(Nadador, Corredor, Ciclista)` — você vai precisar criar `Corredor` e `Ciclista` também. Imprima `Triatleta.__mro__` e explique por escrito, com suas palavras, a ordem que apareceu.

Capítulo 7 — Polimorfismo (visão geral)

Assim como o capítulo 5, este é o resumo. O aprofundamento completo — duck typing, sobrecarga de operadores com dunder methods, ABCs — está no documento separado.

Conceito

Polimorfismo (do grego, “muitas formas”) é a capacidade de **objetos de tipos diferentes responderem à mesma chamada de método de formas diferentes**, permitindo que o código que usa esses objetos não precise saber qual tipo exato está manipulando.

```
class Nadador:
    def treinar(self):
        return "nadando 2km"

class Halterofilista:
    def treinar(self):
        return "levantando peso"

def sessao_de_treino(atleta):
    # essa função não sabe (nem precisa saber) qual classe é 'atleta'
    print(atleta.treinar())

sessao_de_treino(Nadador())          # nadando 2km
sessao_de_treino(Halterofilista())  # levantando peso
```

Repara que Nadador e Halterofilista **nem precisam ter uma superclasse comum** — isso é chamado de *duck typing* (“se anda como pato e grasna como pato...”), e é aprofundado no documento separado.

Analogia

O técnico grita “treinem!” pro time inteiro. Cada atleta interpreta esse comando conforme sua modalidade — o nadador vai pra piscina, o halterofilista vai pro rack. O comando é o mesmo (treinar()), a execução é específica de cada um. O técnico não precisa de um if pra cada tipo de atleta.

Exercícios (prévia)

7.1 — Usando as classes do exercício 6.1, escreva uma função `treino_do_dia(lista_atletas)` que itera sobre uma lista misturando Nadador e Halterofilista e chama `.treinar()` em cada um, sem usar nenhum `if isinstance(...)`.

Capítulo 8 — Abstração e classes abstratas (ABC)

Conceito

Abstração é modelar só o que importa pro problema — esconder complexidade irrelevante. Em Python, isso ganha uma ferramenta concreta com o módulo abc: **classes abstratas**, que definem um “contrato” que subclasses são *obrigadas* a cumprir.

```
from abc import ABC, abstractmethod

class Atleta(ABC):
    def __init__(self, nome):
        self.nome = nome

    @abstractmethod
    def treinar(self):
        """Toda subclasse TEM que implementar isso."""
        ...

    def descansar(self):
        return f"{self.nome} está descansando" # comportamento concreto, herdado normalmente

class Nadador(Atleta):
    def treinar(self):
        return f"{self.nome} nadando"

# Atleta("João") # TypeError: Can't instantiate abstract class Atleta
joao = Nadador("João") # OK — implementou treinar()
```

ABC impede a criação de instâncias da classe base incompleta, forçando quem herda a implementar o contrato. Isso é diferente de simplesmente “não implementar e deixar dar erro depois” — o erro acontece **na hora de instanciar**, não lá na frente quando alguém esquece de chamar o método certo.

Analogia

É o edital de matrícula da academia: “todo instrutor tem que ter certificação em primeiros socorros” — não é opcional, não dá pra contratar (instanciar) alguém que não cumpre esse requisito.

Erros comuns

- Confundir abstração (o conceito, presente em qualquer boa modelagem) com ABC (a ferramenta específica do Python). Você pode fazer boa abstração sem nunca usar abc.
- Esquecer que métodos concretos (não abstratos) numa ABC funcionam normalmente e são herdados — ABC não bloqueia tudo, só o que é marcado `@abstractmethod`.

Exercícios

8.1 — Transforme a `Atleta` do capítulo 6 numa ABC com `treinar()` abstrato. Tente instanciar `Atleta` diretamente e leia o erro completo.

8.2 — Adicione um segundo método abstrato `avaliar_desempenho(self)` e implemente nas duas subclasses existentes.

8.3 (desafio) — Pesquise (documentação oficial do abc) o que é uma `@property` abstrata e implemente uma em `Atleta` chamada `modalidade`, que cada subclasse deve retornar como string.

Capítulo 9 — Métodos mágicos (dunder methods)

Conceito

Métodos com nome tipo `__init__`, `__str__`, `__len__` são chamados de *dunder* (double underscore) ou *métodos mágicos*. Eles conectam suas classes ao comportamento nativo do Python — permitem usar `print()`, `len()`, operadores (`+`, `==`, `<`), `for`, etc. diretamente nos seus objetos.

```
class Atleta:
    def __init__(self, nome, peso):
        self.nome = nome
        self.peso = peso

    def __str__(self):
        """Usado por print() e str() – representação amigável."""
        return f"Atleta: {self.nome} ({self.peso}kg)"

    def __repr__(self):
        """Usado no console/debug – deveria, idealmente, ser reproduzível."""
        return f"Atleta(nome={self.nome!r}, peso={self.peso})"

    def __eq__(self, outro):
        """Define o que significa == entre dois Atletas."""
        return self.nome == outro.nome and self.peso == outro.peso

    def __lt__(self, outro):
        """Define <, o que habilita sorted() automaticamente."""
        return self.peso < outro.peso

joao = Atleta("João", 82)
print(joao)           # Atleta: João (82kg) – usa __str__
print([joao])         # [Atleta(nome='João', peso=82)] – usa __repr__

maria = Atleta("Maria", 65)
print(joao == maria)  # False – usa __eq__
print(sorted([joao, maria])) # ordenado por peso – usa __lt__
```

Tabela de referência rápida

Dunder	Habilita
<code>__init__</code>	Inicialização na criação do objeto
<code>__str__</code> / <code>__repr__</code>	<code>print()</code> , <code>str()</code> , representação no console
<code>__eq__</code> , <code>__lt__</code> , <code>__le__</code> , etc.	<code>==</code> , <code><</code> , <code>sorted()</code> , <code>min()</code> , <code>max()</code>
<code>__len__</code>	<code>len(objeto)</code>
<code>__getitem__</code>	<code>objeto[indice]</code> , iteração se não houver <code>__iter__</code>
<code>__add__</code> , <code>__sub__</code> , etc.	<code>+</code> , <code>-</code> e demais operadores aritméticos
<code>__call__</code>	Permite chamar o objeto como se fosse função: <code>objeto()</code>

Sobrecarga de operadores (`__add__`, `__eq__` etc.) é aprofundada com mais exemplos no documento de Polimorfismo, já que é uma forma de polimorfismo ad-hoc.

Erros comuns

- Implementar `__eq__` sem implementar `__hash__` — objetos com `__eq__` customizado deixam de ser hasháveis por padrão (não dá mais pra usar em set ou como chave de dict) a menos que você defina `__hash__` também.
- Confundir `__str__` (pra usuário final) com `__repr__` (pra debug/desenvolvedor) — boa prática é `__repr__` sempre existir e ser o mais próximo possível de código que recria o objeto.

Exercícios

9.1 — Implemente `__str__` e `__repr__` na sua classe Exercício do capítulo 3.

9.2 — Implemente `__eq__` e `__lt__` em Atleta comparando por peso. Crie uma lista de 4 atletas e use `sorted()` nela.

9.3 (desafio) — Implemente `__len__` em PlanoDeTreino (capítulo 3) retornando a quantidade de exercícios, e `__getitem__` permitindo `plano[0]` acessar o primeiro exercício. Depois de implementar `__getitem__`, teste se `for exercicio in plano:` funciona — e explique por escrito por que funciona mesmo sem você ter implementado `__iter__`.

Capítulo 10 — Composição vs Herança

Conceito

Regra de ouro: **“favoreça composição sobre herança”**. Herança modela “é um”; composição modela “tem um”. Composição costuma ser mais flexível porque evita hierarquias rígidas e profundas, que são difíceis de mudar depois.

```
class FichaDeTreino:
    def __init__(self):
        self.exercicios = []

    def adicionar(self, exercicio):
        self.exercicios.append(exercicio)

class Atleta:
    def __init__(self, nome):
        self.nome = nome
        self.ficha = FichaDeTreino() # Atleta TEM uma FichaDeTreino – composição

    def montar_treino(self, exercicio):
        self.ficha.adicionar(exercicio)

joao = Atleta("João")
joao.montar_treino("Supino")
```

Se fosse herança (`class Atleta(FichaDeTreino)`), você estaria dizendo que “todo atleta É uma ficha de treino” — semanticamente errado, e além disso trava a possibilidade de um atleta ter mais de uma ficha, ou trocar de ficha dinamicamente.

Analogia

Um atleta **tem** um plano de treino, **tem** uma dieta, **tem** um histórico de lesões — várias composições. Ele não **é** um plano de treino. Se você modelasse por herança, ia rapidamente cair em hierarquias artificiais tentando forçar relações que não existem de verdade.

Quando usar cada um

- Use **herança** quando a relação “é um” é genuinamente verdadeira e estável (um Powerlifter sempre será um Atleta).

- Use **composição** quando é “tem um”, ou quando você quer trocar o comportamento em tempo de execução (ex: trocar a ficha de treino do atleta sem recriar o objeto inteiro).

Erros comuns

- Criar hierarquias profundas (Atleta > AtletaDeForca > Powerlifter > PowerlifterRaw) só pra reaproveitar código, quando composição resolveria com muito menos acoplamento.

Exercícios

10.1 — Refatore o exercício 3.2 (PlanoDeTreino) para que Atleta tenha um PlanoDeTreino como atributo, em vez da lista de exercícios estar solta.

10.2 (desafio) — Modele Academia, que **tem** vários Atletas (lista) e **tem** um Endereco (outra classe pequena, com rua, cidade). Justifique por escrito por que nenhuma dessas relações deveria ser herança.

Capítulo 11 — Princípios SOLID (aplicados, sem enrolação)

Conceito

SOLID são 5 princípios de design que ajudam POO a não desandar em código difícil de manter. Aqui vai a versão direta ao ponto, com exemplo mínimo de cada um:

S — Single Responsibility: uma classe deve ter um só motivo pra mudar.

```
# RUIM: Atleta faz cálculo E persistência E formatação
# BOM: separar responsabilidades
class Atleta: ... # dados e regras do atleta
class RepositorioAtleta: ... # salvar/carregar do banco
class RelatorioAtleta: ... # formatar saída
```

O — Open/Closed: aberto para extensão, fechado para modificação. Se adicionar um tipo novo de atleta te obriga a editar um if/elif gigante em algum lugar, você violou isso — polimorfismo (capítulo 7) é a ferramenta que resolve.

L — Liskov Substitution: uma subclasse deve poder substituir a superclasse sem quebrar o comportamento esperado. Se `Powerlifter(Atleta)` sobrescreve `treinar()` de um jeito que quebra o contrato esperado por quem usa `Atleta`, isso é violação de Liskov.

I — Interface Segregation: várias interfaces específicas são melhores que uma interface genérica gigante que força classes a implementar métodos que não fazem sentido pra elas.

D — Dependency Inversion: dependa de abstrações (ex: uma ABC), não de implementações concretas. Isso facilita trocar peças do sistema sem reescrever tudo.

Por que isso importa aqui

Você não precisa decorar as siglas. Precisa reconhecer o cheiro de código que viola cada um, porque esses são exatamente os problemas que os pilares de POO (encapsulamento, herança, polimorfismo, abstração) existem pra resolver na prática.

Exercícios

11.1 — Olhe pro seu `Cleitinho_ChatBot`: existe alguma classe/função fazendo mais de uma coisa (ex: lidar com API E com interface gráfica ao mesmo tempo)? Descreva por escrito como você separaria em responsabilidades únicas.

Capítulo 12 — Projeto integrador

Chegou a hora de juntar tudo. Não tem gabarito pronto pra esse — é prática deliberada de verdade.

Especificação

Construa um sistema de **gerenciamento de treinos** com os seguintes requisitos:

1. Classe abstrata `Exercicio` (ABC) com método abstrato `calcular_volume()` (retorna um número representando o “volume” do exercício — pode ser `series * repeticoes` para exercícios de força, ou `distancia_km` para exercícios de cardio).
2. Pelo menos duas subclasses concretas de `Exercicio`: uma de força (`ExercicioForca`) e uma de cardio (`ExercicioCardio`), cada uma implementando `calcular_volume()` do seu jeito.
3. Classe `Atleta` com encapsulamento correto do peso (usando `@property`, validando que é positivo — consulte o documento de aprofundamento se travar aqui) e composição com uma lista de exercícios feitos.
4. Implemente `__str__`, `__eq__` e `__lt__` em `Atleta` (comparando por volume total treinado).
5. Uma função `ranking(lista_atletas)` que usa `sorted()` — deve funcionar por polimorfismo puro, sem nenhum `isinstance()`.
6. Bônus: um `@classmethod` em `Atleta` chamado `a_partir_de_dict(cls, dados)` que cria um atleta a partir de um dicionário `{"nome": ..., "peso": ...}`.

Como validar seu próprio trabalho

Antes de considerar pronto, responda por escrito:

- Onde eu usei herança? Faz sentido semântico (é um) em cada lugar?
- Onde eu usei composição? Por quê ali e não herança?
- Se eu tentar criar um `Atleta` com peso negativo, o quê acontece? (Deveria estourar erro, não silenciosamente aceitar.)
- Se eu adicionar um terceiro tipo de `Exercicio` amanhã, quanto código existente eu preciso *modificar* (idealmente: zero, só adicionar a classe nova)?

Capítulo 13 — Trilha de estudos e próximos passos

Marco de progresso

Você está aqui:

- [x] Classes e objetos
- [x] Construtores e métodos
- [x] Class methods / static methods
- [~] Encapsulamento <- você parou aqui, aprofundamento no doc separado
- [x] Herança
- [~] Polimorfismo <- próximo, aprofundamento no doc separado
- [x] Abstração / ABC
- [x] Dunder methods
- [x] Composição vs Herança
- [x] SOLID (visão geral)
- [] Projeto integrador — prática, sem prazo fixo, mas não pula

Depois do aprofundamento em Encapsulamento e Polimorfismo, próximos marcos sugeridos

1. **Type hints em POO** (self, nome: str, peso: float -> None) — próximo passo natural pra quem já domina os pilares, ajuda muito na manutenção de projetos como o Library_Dy e o Cleitinho_ChatBot.
2. **Dataclasses** (@dataclass) — reduz boilerplate de __init__ / __repr__ / __eq__ quando a classe é majoritariamente um “saco de dados”.
3. **Design Patterns clássicos aplicáveis em Python** (Strategy, Factory, Observer) — são POO aplicada a problemas recorrentes; Strategy, por exemplo, é literalmente polimorfismo com nome chique.
4. **Testes unitários de classes** (unittest ou pytest) — testar comportamento de objetos, não só funções soltas.
5. Revisitar o Cleitinho_ChatBot e identificar oportunidades reais de aplicar encapsulamento/polimorfismo no código já existente — teoria sem aplicação no seu próprio projeto vale metade.

Reavaliação periódica

Sugestão: depois de terminar o documento de aprofundamento e o projeto integrador, volte aqui e reavalie — o que você consegue explicar sem consultar material? Isso é o teste real de retenção, não “eu lembro de ter lido isso”.

Apêndice — Gabarito comentado

Só olhe aqui depois de tentar de verdade. Gabarito visto antes da tentativa não ensina nada — só dá a ilusão de que você sabia.

Capítulo 2

2.1

```
class Exercicio:
    pass

supino = Exercicio()
agachamento = Exercicio()
remada = Exercicio()
print(supino is agachamento)  # False
print(supino is supino)       # True
```

2.3 — `Atleta.__dict__` mostra os atributos/métodos definidos na classe. `joao.__dict__` só mostra o que foi atribuído *naquela instância* especificamente (via `self.x = ...`). Quando você faz `joao.peso = 90` depois do `__init__`, isso não cria nada novo estruturalmente — só reatribui o valor que já existia no `__dict__` da instância. Se `peso` nunca tivesse sido definido no `__init__`, essa atribuição criaria a chave `peso` no `__dict__` de `joao` pela primeira vez.

Capítulo 3

3.1

```
class Exercicio:
    def __init__(self, nome, grupo_muscular, series, repeticoes):
        self.nome = nome
        self.grupo_muscular = grupo_muscular
        self.series = series
        self.repeticoes = repeticoes

    def resumo(self):
        return f"{self.nome} — {self.grupo_muscular} — {self.series}x{self.repeticoes}"
```

3.2

```
class PlanoDeTreino:
    def __init__(self, nome_atleta):
        self.nome_atleta = nome_atleta
```

```

        self.exercicios = []

    def adicionar_exercicio(self, exercicio):
        self.exercicios.append(exercicio)

    def carga_total(self):
        return sum(e.series * e.repeticoes for e in self.exercicios)

```

Capítulo 4

4.1

```

class Exercicio:
    def __init__(self, nome, grupo_muscular, series, repeticoes):
        self.nome = nome
        self.grupo_muscular = grupo_muscular
        self.series = series
        self.repeticoes = repeticoes

    @classmethod
    def isometria(cls, nome, grupo_muscular, tempo_segundos):
        return cls(nome, grupo_muscular, series=1, repeticoes=tempo_segundos)

```

4.2

```

class Atleta:
    @staticmethod
    def classificar_imc(imc):
        if imc < 18.5:
            return "abaixo do peso"
        elif imc < 25:
            return "peso normal"
        elif imc < 30:
            return "sobrepeso"
        else:
            return "obesidade"

```

Capítulo 6

6.1

```

class Atleta:
    def __init__(self, nome, peso):
        self.nome = nome
        self.peso = peso

    def treinar(self):
        return "treino genérico"

class Nadador(Atleta):
    def treinar(self):
        return "nadando"

```

```
class Halterofilista(Atleta):  
    def treinar(self):  
        return "levantando peso"
```

6.3 — A ordem do `__mro__` segue o algoritmo C3 linearization: basicamente profundidade-primeiro respeitando a ordem em que as superclasses foram listadas, sem repetir uma classe antes de todas as suas subclasses terem sido resolvidas. Na prática, pra herança simples como a do curso, o resultado bate com “a ordem que você escreveu entre parênteses”.

Capítulo 7

7.1

```
def treino_do_dia(lista_atletas):  
    for atleta in lista_atletas:  
        print(atleta.treinar())
```

```
treino_do_dia([Nadador("Ana", 60), Halterofilista("Beto", 85)])
```

Capítulo 9

9.3 — `for exercicio in plano`: funciona mesmo sem `__iter__` porque Python tem um mecanismo de fallback: se o objeto não define `__iter__` mas define `__getitem__`, o `for` tenta acessar `plano[0]`, `plano[1]`, `plano[2]`... incrementando o índice até estourar `IndexError`, que é interpretado como “fim da iteração”. É um comportamento legado, útil de saber que existe, mas em código novo prefira implementar `__iter__` explicitamente quando a iteração é uma parte central do design da classe.